

Unit 1 Answers

5 Marks

1. Explain map, reduce and filter method of array with example.

These are powerful array methods used to iterate over and manipulate data:

- **map()**: Creates a completely new array by applying a function to every element of the original array. It is highly useful for transforming data.
 - *Example:* `let doubled = [1, 2, 3].map(x => x * 2); // Result: [2, 4, 6]`
- **filter()**: Creates a new array containing only the elements that pass a specific true/false condition. It is perfect for searching or removing items.
 - *Example:* `let evens = [1, 2, 3, 4].filter(x => x % 2 === 0); // Result: [2, 4]`
- **reduce()**: Reduces an entire array down to a single value (like a sum or average) by executing a reducer function on each element.
 - *Example:* `let sum = [1, 2, 3].reduce((total, current) => total + current, 0); // Result: 6`

2. Explain spread and rest operator with example.

Both use the exact same ... syntax, but they serve opposite purposes:

- **Spread Operator**: "Spreads" or expands elements of an array (or object) into individual items.
 - *Example:* `let arr1 = [1, 2]; let arr2 = [...arr1, 3, 4]; // Result: [1, 2, 3, 4]`
- **Rest Operator**: Collects multiple individual elements and condenses them into a single array. It is most often used in function parameters to handle an unlimited number of arguments.
 - *Example:* `function sum(...numbers) { return numbers.reduce((a, b) => a + b); }`

3. Explain the promise object with then and catch method.

A **Promise** is an object representing the eventual completion (or failure) of an asynchronous operation (like fetching data from a server). It prevents the browser from freezing while waiting.

- **.then()**: This method executes when the promise is successfully fulfilled.
- **.catch()**: This method executes if the promise fails or is rejected.
- *Example:*

```
let task = new Promise((resolve, reject) => resolve("Success!"));
```

```
task.then(result => console.log(result)).catch(error => console.log(error));
```

4. Explain the purpose of **async** and **await**.

The **async/await** syntax is a modern way to handle Promises. Its purpose is to make asynchronous code look and behave like synchronous, line-by-line code, preventing messy "callback hell".

- **async**: Placed before a function, ensuring it automatically returns a Promise.
- **await**: Pauses the execution of the function until the Promise resolves, making it much easier to read without chaining `.then()`.
 - *Example:*

JavaScript

```
async function getData() {  
  let response = await fetch('url');  
  let data = await response.json();  
}
```

3 Marks

1. Differentiate between **let** and **const** with suitable example.

- **let**: Modern JavaScript uses `let` for values that might change over time, like a player's running score.
- **const**: Use `const` for values that are locked in and permanent, like a fixed configuration setting.
 - *Example:* `let score = 10; score = 15; // Valid update!. const username = "Admin"; username = "User"; // Throws an error.`

2. Explain arrow function with suitable example.

An **Arrow Function** is a shorter, modern syntax for writing functions. They handle context differently than regular functions, particularly when dealing with the `this` keyword. They do not bind their own `this` context.

- *Example:* `const add = (a, b) => a + b;`

3. Explain how **set** can be used to remove the duplicate values from an array.

A **Set** is a built-in object that strictly stores only unique values. By passing an array with duplicates into a new `Set`, duplicates are instantly filtered out. You can then use the spread operator (`...`) to turn the `Set` back into a normal array.

- *Example:* `let unique = [...new Set([1, 2, 2, 3])]; // Result: [1, 2, 3]`

4. Explain import and export in javascript.

They are used to share code (modules) between different files.

- **export:** Makes a function, variable, or class available so other files can use it.
- **import:** Brings the exported code from another file into your current working file.
 - *Example:* export const pi = 3.14; in one file, and import { pi } from './math.js'; in another.

1 Marks

1. **What is the full form of ECMAScript?:**
European Computer Manufacturers Association Script (the official standard for JavaScript).
2. **What is ternary operator?:**
A quick shortcut for simple true/false variable assignments.
Example: let access = (age >= 18) ? "Granted" : "Denied";.
3. **What is template literals?:**
Strings wrapped in backticks (``) that allow embedded expressions using \${variable}.
4. **What is a constructor?:**
A special initialization method inside a class that runs automatically when a new object is created.
5. **What is a promise?:**
An object representing the eventual completion or failure of an asynchronous operation.

Unit 2 Answers

5 Marks

1. Explain mounting phase of lifecycle of class component.

The **Mounting phase** occurs when a component is created and inserted into the DOM for the first time. The lifecycle methods run in this exact order:

1. **constructor()**: Initializes the component's state and binds event handlers.
2. **static getDerivedStateFromProps()**: Syncs state with initial props right before rendering.
3. **render()**: The only required method. Returns the JSX/HTML to construct the UI.
4. **componentDidMount()**: Runs immediately after the component is rendered into the DOM. This is the perfect place to make API calls or set up subscriptions.

2. Explain useEffect Hook.

The **useEffect** Hook lets you perform side effects in functional components (like fetching data, subscribing to events, or manipulating the DOM). It effectively replaces the need for `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount` from class components. It accepts a function and an optional dependency array to control when the effect runs.

- *Example:*

JavaScript

```
useEffect(() => {  
  console.log("Component mounted or updated!");  
  return () => console.log("Cleanup on unmount");  
}, []);
```

3. Difference between class and function component.

- **Class Components:** Use ES6 classes, require a `render()` method, manage state using `this.state`, and have complex lifecycle methods (like `componentDidMount`). They rely heavily on the `this` keyword.
- **Function Components:** Simpler JavaScript functions that return JSX. React requires all custom components to start with a capital letter. They use React Hooks (like `useState` and `useEffect`) for state and lifecycle management. They are shorter, easier to read, and do not use the `this` keyword.

4. Explain updating phase of lifecycle of class component.

The **Updating phase** happens when a component re-renders due to changes in its props or state. The sequence is:

1. **static getDerivedStateFromProps():** Updates state based on new incoming props.
2. **shouldComponentUpdate():** Returns true/false to decide if React should skip the render process (useful for performance optimization).
3. **render():** Re-renders the UI with the updated data.
4. **getSnapshotBeforeUpdate():** Captures old DOM details (like scroll position) right before the DOM updates.
5. **componentDidUpdate():** Runs after the update has been fully applied to the DOM.

3 Marks

1. Explain useState hook.

The **useState** hook allows you to track state in a function component. It returns an array containing exactly two items: the current state value, and a dedicated function to update that state.

- *Example:* `const [username, setUsername] = useState("");`

2. Explain the function component with props (passing and accessing).

Props (properties) are read-only data passed from a parent component down to a child component.

- **Passing:** Add custom attributes to the component tag. `<Greeting name="Amit"/>`
- **Accessing:** The function component accepts props as its first argument.
 - *Example:* `function Greeting(props) { return <h1>Hello {props.name}</h1>; }`

3. How to pass default value to props.

The most modern and common way to pass default props is to assign default values directly inside the function parameters using JavaScript object destructuring.

- *Example:* `function Profile({ name = "Guest" }) { return <p>Welcome {name}</p>; }`

4. Explain Children props in function component.

`props.children` is a special prop automatically passed to components that have opening and closing tags. It contains whatever elements are placed between those tags, making it perfect for creating reusable layout wrappers like Cards or Modals.

- *Example:* `function Card(props) { return <div className="card">{props.children}</div>; }`

1 Marks

1. **Which method is called when component is unmount?:**
`componentWillUnmount()`
2. **What is JSX?:**
JavaScript XML. A syntax extension for React that allows you to write HTML structures directly inside JavaScript code.
3. **What is state object in React?:**
A built-in React feature used to store dynamic data or information about the component that can change over time.
4. **What is the flow of state in react?:**
Unidirectional (Top-Down) data flow, meaning state only flows downwards from parent components to their child components.
5. **What is the return type of `useState`?:** An array containing exactly two elements: the current state variable and its corresponding state-updating function.

Unit = 2

[1 Mark] Questions & Answers

1. What is JSX? [IMP]

- **Answer:** JSX (JavaScript XML) is a syntax extension for JavaScript used in React to describe UI components using an HTML-like structure.

2. Which tool converts/transpiles JSX into pure JavaScript?

- **Answer:** Babel (it converts JSX into React.createElement() calls).

3. What naming convention must be followed for React component files and functions? [IMP]

- **Answer:** Component file names and function names must start with a capital letter (PascalCase).

4. What are props in React? [IMP]

- **Answer:** Props (short for properties) are read-only inputs passed from a parent component to a child component (unidirectional data flow).

5. What is props.children? [IMP]

- **Answer:** props.children is a special prop that automatically captures and displays whatever content or JSX is placed between the opening and closing tags of a component.

6. Which hook is used to handle state management in functional components? [IMP]

- **Answer:** The useState Hook.

7. Why do we use unique key attributes when rendering lists in React? [IMP]

- **Answer:** Keys help React identify which list items have changed, been added, or removed, improving re-rendering efficiency.

8. What is the purpose of e.preventDefault() in React event handling? [IMP]

- **Answer:** It prevents the browser's default behavior for an event, such as preventing a form submission from refreshing the entire page.

9. What keyword in CSS Modules is used to inherit styles from another class?

- **Answer:** The composes keyword.

10. What is <Outlet/> in React Router?

- **Answer:** <Outlet/> is a placeholder component that renders matching child route components inside a parent layout component.

11. How are inline styles written in React?

- **Answer:** They are written using the style prop, which accepts a JavaScript object with property keys in camelCase (e.g., style={{ backgroundColor: 'red' }}).

12. Which method is used to update state in a React class component?

- **Answer:** this.setState().

1-Mark Questions & Answers

- **What is JSX in React? [IMP]**

A syntax extension for JavaScript used to describe UI components using an HTML-like structure.

- **What is the Virtual DOM?**

A lightweight JavaScript object copy of the real DOM used by React to compare and update only the changed parts.

- **Why use CSS Modules in React?**

They locally scope CSS by generating unique class names to prevent styling conflicts.

- **What is a React component? [IMP]**

A reusable, independent JavaScript function or class that allows developers to break down the UI into smaller, manageable parts.

- **What are props? [IMP]**

Read-only properties used to pass data in a unidirectional flow from a parent component to a child component.

- **What is the role of props.children?**

It allows you to pass and access JSX content nested between the opening and closing tags of a custom component.

- **What is the state object used for? [IMP]**

It is used to store and manage component-specific data that, when updated (via setState or useState), triggers a UI re-render.

- **What is conditional rendering?**

The process of dynamically showing different UI elements based on a specific condition, often using ternary (?:) or short-circuit (&&) operators.

- **Why are "keys" necessary when rendering lists? [IMP]**

Keys help React uniquely identify which list items have changed, been added, or removed, which improves performance by preventing unnecessary re-renders.

- **How does React handle events differently than standard HTML?**

React uses camelCase naming for events (e.g., onClick) and utilizes a cross-browser wrapper called SyntheticEvent.

- **Why use e.preventDefault() in an event handler? [IMP]**

It stops the default browser behavior, such as preventing the entire page from refreshing when a form is submitted.

- **What is the purpose of the <Outlet/> component?**

It acts as a placeholder that renders child components inside a parent layout component.

Unit - 2

This chapter provides a comprehensive introduction to React, starting with JSX, a syntax extension that allows developers to write HTML structures directly within JavaScript. It explains how React utilizes a component-based architecture to break down user interfaces into reusable, independent functions or classes. You will learn how to pass data dynamically between these components using read-only properties called props, and how to manage a component's internal, changeable data using state. The text also covers essential styling techniques, including inline styles, traditional CSS files, and modular CSS for locally scoped styling. Furthermore, it explores dynamic UI updating techniques such as conditional rendering using ternary and short-circuit operators. You will also discover how to efficiently render collections of data using map functions and unique keys to optimize performance. Finally, it details how React handles user interactions through camel-cased synthetic events and introduces structural routing placeholders like the <Outlet/> component.

Unit = 1

This chapter provides a comprehensive introduction to React, starting with JSX, a syntax extension that allows developers to write HTML structures directly within JavaScript. It explains how React utilizes a component-based architecture to break down user interfaces into reusable, independent functions or classes. You will learn how to pass data dynamically between these components using read-only properties called props, and how to manage a component's internal, changeable data using state. The text also covers essential styling techniques, including inline styles, traditional CSS files, and modular CSS for locally scoped styling. Furthermore, it explores dynamic UI updating techniques such as conditional rendering using ternary and short-circuit operators. You will also discover how to efficiently render collections of data using map functions and unique keys to optimize performance. Finally, it details how React handles user interactions through camel-cased synthetic events and introduces structural routing placeholders like the <Outlet/> component.

1-Mark Questions & Answers

- **What is JSX in React? [IMP]**
A syntax extension for JavaScript used to describe UI components using an HTML-like structure.
- **What is the Virtual DOM? A lightweight JavaScript object copy of the real DOM used by React to compare and update only the changed parts.**
- **Why use CSS Modules in React? They locally scope CSS by generating unique class names to prevent styling conflicts.**
- **What is a React component? [IMP] A reusable, independent JavaScript function or class that allows developers to break down the UI into smaller, manageable parts.**
- **What are props? [IMP] Read-only properties used to pass data in a unidirectional flow from a parent component to a child component.**

- **What is the role of `props.children`? It allows you to pass and access JSX content nested between the opening and closing tags of a custom component.**
- **What is the state object used for? [IMP] It is used to store and manage component-specific data that, when updated (via `setState` or `useState`), triggers a UI re-render.**
- **What is conditional rendering? The process of dynamically showing different UI elements based on a specific condition, often using ternary (`? :`) or short-circuit (`&&`) operators.**
- **Why are "keys" necessary when rendering lists? [IMP] Keys help React uniquely identify which list items have changed, been added, or removed, which improves performance by preventing unnecessary re-renders.**
- **How does React handle events differently than standard HTML? React uses camelCase naming for events (e.g., `onClick`) and utilizes a cross-browser wrapper called `SyntheticEvent`.**
- **Why use `e.preventDefault()` in an event handler? [IMP] It stops the default browser behavior, such as preventing the entire page from refreshing when a form is submitted.**
- **What is the purpose of the `<Outlet/>` component? It acts as a placeholder that renders child components inside a parent layout component.**